ChatGPT

Systematic Visual Diagram Quality Validation Pipeline

Introduction

In complex technical and business environments, **visual diagrams** must meet high quality standards to effectively convey information to stakeholders. This research outlines a **systematic, deterministic pipeline** for validating and improving diagram quality. Rather than relying on subjective AI judgments, the approach enforces measurable design principles and accessibility standards. The goal is to support **any diagram format** (from SVG to PDF or Mermaid code) and produce **high-fidelity outputs** (2× retina PNG at 300 DPI) suitable for professional presentations. All analysis runs in a **consistent Linux CI environment** to ensure deterministic results, with a lightweight container (<500 MB) and fast execution (<5 seconds per diagram) for real-time feedback during creation. The following sections detail how the pipeline ingests various inputs, normalizes output, checks against design rules (contrast, typography, layout, etc.), and provides **actionable improvement recommendations**.

Input Formats and Rendering

Primary input types include SVG illustrations, HTML/CSS-based visuals, PDF diagrams, programmatic charts (e.g. Mermaid.js), and other vector/image formats. The pipeline implements a **universal rendering step** to convert any input into a standardized **PNG image**. This ensures that all downstream analyses work on a consistent raster canvas regardless of source format. For example:

- SVG and Mermaid diagrams can be rendered using headless libraries (e.g. Cairo or librsvg) into high-resolution PNG.
- HTML/CSS or web-based diagrams are rasterized via a headless browser engine in the Linux container.
- PDF pages are converted to PNG using a PDF renderer.

By normalizing to a PNG, the system can apply pixel-based inspections (for contrast, color usage, etc.) uniformly. The output resolution is set to **300 DPI** (dots per inch) at 2× retina scale, yielding **crisp images** suitable for print and projection. This high resolution prevents blurriness when diagrams are enlarged in executive slide decks, aligning with professional presentation standards. (300 DPI is commonly used for print-quality graphics to ensure clarity).

All rendering is done in a **controlled CI environment** to guarantee consistency. The Linux container includes required fonts and rendering libraries so that the appearance of text, colors, and layout remains identical on every run. This deterministic setup avoids issues where a diagram might look different on another machine. The container footprint is kept under 500 MB by using minimal base images and specialized libraries (e.g. Alpine Linux with only the necessary rendering tools). Despite the small footprint, the pipeline can process each diagram in under **5 seconds**, enabling near real-time feedback as a user edits or submits a diagram.

Quality and Performance Requirements

High output quality is critical because stakeholders often make decisions based on these visuals. The pipeline's PNG outputs use a **2× retina scale** (e.g. rendering at double the target dimensions) and 300 DPI so that when placed in presentations or documents, the diagrams appear sharp on high-density displays and in print. This prevents pixelation when zooming or on large screens, delivering a professional impression.

The **runtime performance** (<5 seconds per image) ensures the tool can be used interactively. For instance, a diagramming application can call the validation pipeline on-the-fly, giving the creator immediate feedback without interrupting their flow. To achieve this speed, the system uses optimized algorithms and possibly pre-computed data (such as caching standard assets or models). Parallel processing and efficient image analysis techniques are employed so that even moderately complex diagrams meet the time budget. If a diagram is extremely complex, the system might degrade gracefully (e.g. analyzing a subset of rules first or using approximations) to still return quick insights.

Environment consistency is also a quality factor. By running all analyses in a CI (Continuous Integration) environment (e.g. a Docker container on Linux), results do not vary between users or sessions. Deterministic behavior is ensured by pinning library versions and using the same configuration every time. This means if a diagram passes the quality checks once, it will continue to pass as long as the content is unchanged – a vital feature for regression testing visual outputs in an automated pipeline.

Universal Visual Validation Pipeline

The core of the system is a **universal visual validation pipeline** that can be summarized in three stages:

1. **Input Ingestion & Normalization:** The raw input (SVG, HTML, PDF, etc.) is parsed and rendered to a high-resolution PNG image. During this step, any platform-specific elements (fonts, CSS features) are handled by the headless renderer to mirror actual appearance. The result is a pixel-perfect representation of the diagram as an image.

2. **Systematic Quality Analysis:** The PNG output is then subjected to a series of **deterministic checks** against predefined quality rules. Rather than "eyeballing" aesthetics, the system uses **measurable criteria** derived from design standards and accessibility guidelines. The analysis covers multiple aspects – color contrast, font sizes, spacing consistency, flow logic, etc. – detailed in the next sections. Each check yields quantifiable metrics (e.g. contrast ratio values, font size in points, count of colors used) which are compared against thresholds or rules. Any deviation or violation is flagged.

3. **Improvement Recommendations:** Based on the analysis findings, the pipeline generates **specific, actionable feedback** for the user. Instead of vague critiques, it provides concrete suggestions to fix issues. For example, if a text label's color fails contrast standards, it might recommend increasing the contrast by darkening the text color or lightening the background. If too many colors are used, it could suggest merging some categories or using a consistent palette. These recommendations are formulated in clear language, each tied to a measurable rule (e.g. "Font size is 10pt, below the 12pt minimum – increase text size for readability"). The output can be a structured report or annotated image highlighting problem areas.

This **universal pipeline** design ensures that regardless of input format or content, every diagram goes through the same rigorous quality lens. By converting all inputs to a common form (bitmap image) and applying objective rules, the system achieves **cross-format compatibility** – an SVG exported from design software and a PDF from a BI tool will be judged by identical standards. This creates a **level playing field** for quality validation.

Deterministic Quality Rules and Checks

At the heart of the pipeline are **deterministic quality rules** that encapsulate proven design principles and accessibility criteria. Each rule is implemented as an automated check that yields a binary pass/fail or a numeric score. Together, these rules cover the critical aspects of a diagram's readability, accuracy, and aesthetics. Below we outline the major categories of checks and their significance:

Color Contrast and Accessibility

Ensuring sufficient **contrast** between text (or important graphical elements) and background is crucial for legibility, including for users with low vision or color vision deficiencies. The pipeline measures the **luminance contrast ratio** for all text and key shapes in the diagram. According to the Web Content Accessibility Guidelines (WCAG) Level AA, **normal text should have at least a 4.5:1 contrast ratio** against its background [1]. Our system checks each text label and shape outline against this threshold. For example, white text on a pale yellow box would likely fail (as white/yellow contrast is low), triggering a recommendation to darken the text or use a darker background [2]. Non-text graphical elements that are essential for understanding (like chart lines or flow connectors) are similarly expected to meet contrast minimums of 4.5:1 [3], unless they are purely decorative. This deterministic rule guarantees that **every diagram is perceivable** by people with moderately low vision [4].

In addition to contrast, **colorblind accessibility** is verified. The pipeline analyzes the color palette to detect any problematic color combinations, especially the notorious **red-green pairing**. Red-green distinctions are the most common difficulty for colorblind individuals [5]. If a diagram uses red and green as the sole means to differentiate elements, the system will flag this. The recommendation might be to choose an alternative palette (e.g. blue vs. orange) or add patterns/textures to differentiate elements beyond just color [6] [7]. The validation ensures that color is **not the only channel** for critical information – aligning with best practices to always provide a secondary cue (like labels or icons) for color-coded content [6].

Furthermore, the pipeline can simulate colorblind vision on the image (using known filters for protanopia/deuteranopia, etc.) to programmatically verify that all information is still discernible. If certain categories become indistinguishable in the simulation, that's reported as a flaw. By enforcing these checks, the system upholds **accessible design** so that diagrams are usable by the widest audience.

Color Palette Limit and Consistency

Effective diagrams typically use a **limited color palette** to avoid visual overload and to make it easy for viewers to remember what each color signifies. Our quality rules enforce that the number of distinct colors stays within a reasonable limit (approximately 5 to 7 colors maximum). In fact, research on graph legibility suggests using *no more than 5 colors* if the viewer is expected to differentiate or remember the categories represented [8]. Using too many hues can confuse readers, as differences become harder to discern [9]. The pipeline will count the unique colors (accounting for small variations) in the diagram. If it exceeds the

threshold, it flags an issue. The recommendation might be to combine similar categories or use textures to reduce reliance on color differences.

Equally important is **color consistency** across the diagram. If the same concept or category appears in multiple places, the pipeline checks that the color used is consistent throughout (e.g. if "Process A" is blue in one part of the diagram, it should not randomly appear as green elsewhere). This standard follows the principle of using **consistent visual elements** for meaning [10]. In practice, the system can map text labels or legend entries to colors; if a label maps to two different colors, it's a consistency error. By enforcing a consistent palette, the diagram not only looks more professional but also prevents misinterpretation.

Finally, if the diagram uses corporate or brand colors, the pipeline can verify those against an allowed list to ensure brand compliance. Any off-brand colors or inconsistent shades would be identified. The overall **color system validation** guarantees a balanced palette: a limited number of colors, no disallowed combinations (like red vs. green for critical distinctions), and uniform usage across the graphic.

Typography and Legibility

Clear **typography** is a non-negotiable part of diagram quality. All text in the diagram – titles, labels, annotations – must be easily readable at the intended viewing size. The pipeline examines text properties such as **font size**, font style, and spacing. A minimum font size threshold (commonly **12 point** for body text) is enforced to avoid tiny, illegible labels [11]. Many accessibility and design guidelines recommend 12 pt (approximately 16px on screen) as a baseline for readable text on digital materials [11]. If any detected text falls below this size, the system flags it. The fix might be to increase the font size or, if space is an issue, to simplify or abbreviate the text content while staying above the minimum size.

The pipeline also ensures **font consistency** and appropriate styling. Mixing too many typefaces or styles can look unprofessional and distract from content. The rule is to use a consistent font (or a harmonious set of fonts for headings and body) throughout the diagram [10]. If the system finds multiple font families or an inconsistent use of bold/italics without purpose, it will suggest tightening the typography. For example, all shape labels might need to use the same sans-serif font for a clean look. This echoes the advice to *"stick to the same typography… throughout your diagram"* [10] unless a variation carries meaning (like italicizing all notes).

Another legibility factor is **text spacing and placement**. The pipeline checks that labels are not overcrowded or overlapping with other elements. Each text element's bounding box is analyzed to ensure it has enough padding from shape borders and other text. If a label is found running into the edge of a shape or colliding with another label, it's flagged for adjustment (e.g. increasing the shape size or wrapping text). The system might also verify that text is not placed at hard-to-read angles; where possible, horizontal text is preferred for readability.

By imposing these typography rules (minimum size, consistency, clarity), the system ensures that *every piece of text can be comfortably read* by the target audience, avoiding issues like squinting or misreading in presentations. Good typography also enhances the overall professional appearance of the visual.

Layout, Spacing, and Flow

Diagrams need a logical **visual flow** so that viewers can follow the information path with ease. The pipeline evaluates the **layout structure** of the diagram, including element alignment, spacing, and directional flow. One fundamental principle it checks is that flows are organized in a **left-to-right or top-to-bottom reading order**, consistent with how readers naturally scan content [12] [13]. If the input diagram is a flowchart or process diagram, the system will verify that the sequence of steps proceeds in a clear direction (e.g. the start node is at the left/top and the end at the right/bottom). A violation would be an arrow that goes *right-to-left* against the main flow or an illogical back-and-forth layout. In such cases, the recommendation is to rearrange components to maintain a straightforward flow direction (for Western audiences, typically left-to-right) [13]. This ensures **clear entry and exit points** – e.g. having a distinct start and end – so stakeholders immediately recognize where the process begins and ends.

Proper **grouping and spacing** of elements is another check. The pipeline identifies clusters of related shapes (perhaps by analyzing connection lines or proximity) and sees if they are visually grouped together with adequate separation from other clusters. Overlapping or tightly packed nodes can make the flow "difficult to follow" [14]. For example, if decision diamonds and their outcome branches are too close to other steps, the system might suggest adding whitespace or using swimlane containers to delineate groups. Consistent spacing between symbols is also measured [12] – the distances between similar shapes should not vary wildly. The system might compute the average gap between flowchart nodes and highlight any large outliers that break the rhythm.

Alignment plays a big role in a professional look. The pipeline checks that shapes that should be aligned (horizontally or vertically) are properly aligned to a common axis. Misalignments (e.g. one process box slightly offset from others in a column) will be detected by comparing their coordinates. The rule is that elements in a sequence or at the same level should line up neatly in rows or columns. According to design tips, maintaining *"consistent spacing between objects and vertical and horizontal alignment"* makes a diagram cleaner and easier to understand [15]. If misalignment is found, the suggestion would be to use alignment guides or tools to straighten the layout.

The pipeline also confirms that **connectors** (lines/arrows) are well-behaved: not crossing unnecessarily, not forming confusing angles, and clearly connecting to the center of shapes. If a connector overlaps text or another connector, the system flags it as a clarity issue (with a tip to reroute or use elbow connectors to avoid overlaps). Ensuring connectors are "prominent enough" and not overly long or winding is another derived rule, because tangled lines can confuse readers [14].

By enforcing layout and flow rules, the pipeline helps produce visuals that have a clear hierarchy and path. Viewers will find it easy to start at the intended point, follow the sequence through logically, and not get lost due to chaotic placement. These checks reduce cognitive load by aligning with best practices like **logical flow, consistent formatting, and modular organization** (e.g. breaking large diagrams into sub-sections) [16] [17].

Standard Symbols and Notation

To avoid confusion, diagrams should adhere to **conventional notation** where possible. The pipeline verifies the use of **standard symbols** in certain diagram types. For instance, in flowcharts a **diamond shape** typically represents a decision point. If the input format provides shape information (e.g. a Mermaid

flowchart, or an SVG with metadata), the system will check that decisions, start/end points, processes, etc., use appropriate shapes or icons. Using the correct symbol for each step is one of the basic flowchart rules [18]. Should it detect a non-standard shape (say, using a circle where a decision is expected), the tool will warn the user and suggest the standard symbol. Consistency in notation ensures that any knowledgeable reader can interpret the diagram quickly.

Moreover, the pipeline encourages inclusion of a **legend or key** when non-standard or varied symbols are used. The Nulab flowchart guide notes that if you do adopt a different convention, providing a key is essential for understanding [19]. Our system might flag the absence of a legend if it sees multiple custom shapes or colors that aren't self-evident, reminding the author to include a brief explanation for the audience's sake.

Another aspect of notation consistency is using uniform arrow styles and line styles. The pipeline checks that all arrows in a diagram look the same (unless representing different relationships, in which case a legend is needed). Mixed line styles (dashed vs solid) or different arrowhead types should have meaning behind them. If not, the system will recommend standardizing them to one style to avoid implying distinctions that do not exist. This aligns with maintaining *consistent visual elements throughout the diagram* [10] and only introducing variation when it conveys information (e.g. dashed line for an optional connection, whereas the legend explains it).

Finally, text notation like labeling conventions are examined. For example, in an architecture diagram, if components are labeled with IDs in one section but names in another, it's inconsistent. The tool could point this out and suggest using a single labeling scheme. By **enforcing notation standards**, the pipeline upholds clarity and professionalism, making diagrams easier to interpret correctly.

Model and Data Consistency Verification

A more advanced feature of the pipeline is **model-diagram consistency** checking. Many diagrams are visual representations of an underlying data model, process description, or specification. The pipeline can be extended to cross-verify the diagram against a source-of-truth if available. For instance, if a UML class diagram is generated from a data model, the system can parse the model (perhaps provided in a file or via an API) and ensure the diagram contains all the elements it should and no incorrect ones. In general, model consistency validation might include checks like:

- **Completeness:** Every key entity or step from the source model appears in the diagram. If something is missing (e.g. one module of a system isn't shown), the report will note that the visual may be incomplete.
- **No Extraneous Elements:** Conversely, if the diagram has elements that don't map to any part of the known model or spec, those might be accidental and could confuse viewers. The system flags such elements as potential stray or outdated items.
- **Consistency of Names/Values:** The text in the diagram (like component names, data fields, percentages in a chart) is compared to the source data. Any discrepancies (e.g. a value changed in the database but not updated on the chart) are highlighted so the diagram always **matches the latest data**.

Implementing this requires integration with the model data. In a deterministic pipeline, this could mean the user provides a reference JSON or XML of the model, or the tool has modules for specific formats (like

reading a Mermaid sequence diagram definition and ensuring no steps are skipped). Though this check may not always apply (for user-drawn diagrams without formal models, the system can't magically infer correctness), it is crucial in contexts where accuracy is paramount. The pipeline, when used in a CI/CD process, can serve as a guardrail to catch diagrams that have fallen out of sync with the systems or processes they document.

The **benefit of model consistency checks** is confidence that visuals are not just pretty but also truthful. Stakeholders can trust that what they see reflects the actual system or data. If a mismatch is found, the recommendation is clear: update the diagram or the documentation so they align. In essence, it promotes a "single source of truth" practice.

Stakeholder-Appropriate Output

Not all audiences have the same needs, so the pipeline incorporates rules to adjust diagram quality based on the intended **stakeholder audience**. Internal technical documentation might tolerate more complexity or dense information, whereas an executive briefing demands simplicity and polish. It's been noted that *different diagrams for different audiences is the key point* in effective communication [20]. Our system can operate in different modes or profiles (e.g. "Internal Review" vs "Executive Presentation") that tweak certain thresholds and recommendations:

- **Executive/High-Level Audience:** In this mode, the validation is stricter about clarity and brevity. It might enforce a lower maximum number of shapes on a diagram (to avoid overwhelming busy executives), larger minimum font size (since it may be viewed from a distance in a meeting room), and a more visually engaging style. For example, it could encourage inclusion of the company logo or brand colors and discourage very detailed legends or tiny text. High-level stakeholders often prefer diagrams that are *"more entertaining and alive"* to keep their attention [21]. Thus, the pipeline might even flag a diagram that is technically correct but too "dull," suggesting the use of a bit more color or imagery for engagement (without sacrificing substance). It will also emphasize removing any extraneous detail that isn't relevant to the main point, ensuring a concise story is told.

- **Technical/Detail-Focused Audience (Internal):** In this mode, completeness and accuracy take precedence. The pipeline might allow more elements on the diagram, assuming the viewer is willing to spend time on it. It will still enforce core legibility and consistency rules, but might not require the same level of visual flash. In fact, overly decorative elements could be flagged as unnecessary. For instance, an engineer might want all components labeled and all relationships shown, even if the diagram gets busy. The pipeline would focus on making sure that detail is presented in a structured way (perhaps suggesting breaking into sub-diagrams if it's too large, rather than removing detail). As one source notes, a highly detail-focused stakeholder finds *"pretty diagrams can be counterproductive, especially if prettiness comes at the expense of information content"* [22]. So the system ensures substance is retained for this audience.

In practice, the user (or an integrated context in the tool) would specify the target audience or document type. The pipeline's recommendation output would then be tuned accordingly. For example, if a diagram intended for executives has small text or too many technical acronyms, it would warn that those might not be understood or seen, and recommend simplification. Conversely, for an internal diagram, it might be acceptable to use domain-specific jargon and normal contrast (assuming viewers are at their desks), so some checks could be relaxed.

This **audience-aware validation** helps creators adapt their diagrams for maximum impact. It aligns with communication best practices: fit the **presentation to the audience** rather than a one-size-fits-all approach [20]. By automating this, the pipeline serves as a safeguard that the diagram one shows to top management is suitably polished and high-level, whereas the one for the engineering team is detailed and precise – each without quality flaws in their respective context.

Real-Time Feedback and Automation

A key aspect of this solution is providing **real-time feedback** to users during diagram creation. The entire pipeline is optimized for speed so that it can be integrated into interactive tools. For instance, as a user draws a shape or changes a color, the pipeline could re-run on the fly (or on-demand with a "Check Quality" button) and update the list of suggestions almost instantly. This immediate feedback loop makes the quality enforcement feel like a helpful assistant rather than a burdensome afterthought.

From an implementation standpoint, the pipeline can run as a local service or a cloud microservice that the diagramming software calls. It takes the current diagram state as input, runs the analysis (<5 seconds), and returns a structured list of issues and recommendations. The user interface can then highlight these issues on the diagram (e.g. red outline around a low-contrast text box, with a tooltip suggestion to darken the text). By seeing problems highlighted in context, users can quickly iterate – adjust the color, re-run the check, and see the issue resolved.

This approach strongly emphasizes **measurable rules** in feedback. Instead of saying "This looks a bit off," it might say "Arrow from *Task B* goes right-to-left against flow direction – consider reversing it for left-to-right consistency." Each suggestion is tied to a specific guideline, making it easy to understand and fix. Over time, users learn the principles as well, improving their own design skills.

The pipeline's automation also fits into continuous integration (CI) for documentation. Teams can include diagram quality checks as part of their CI pipeline: if someone commits a diagram that doesn't meet standards, the system can flag it or even fail the build with the listed issues. This is analogous to code linters or test cases, but for visuals. It ensures that only diagrams that meet the agreed-upon quality bar are published internally or externally.

Even with automated enforcement, the tone of recommendations remains **supportive and specific**. Each issue is accompanied by a suggestion on how to improve. For example: "Contrast ratio for label 'Step 1' is 3:1, below 4.5:1 – increase font weight or use a darker text color for better readability." Or "Diagram uses 10 distinct colors – try limiting to ~5 for clarity [8], reusing colors or using patterns to differentiate similar items." By providing a concrete fix, the system avoids leaving the user guessing. It also steers clear of subjective language; every recommendation can be traced to a rule (contrast ratio, count of colors, font size number, etc.), reinforcing the **deterministic nature** of the evaluation.

Cross-Format Compatibility and Consistency

Because the pipeline treats everything after rendering as an image, it is inherently **format-agnostic** in its analysis phase. Whether the diagram originated in PowerPoint, a code-based tool, or drawn by hand and scanned (if converted to an image), the same quality metrics apply. This consistency is important for organizations that have multiple diagram sources – you might have engineers making architecture

diagrams in a text-based DSL and business analysts making flowcharts in Visio. With our system, both end up being PNGs that get the same battery of tests. This unified approach ensures a **baseline quality standard** across the board.

However, where the input format *can* be leveraged for more precision, the pipeline does so. For example, if analyzing an SVG or a Mermaid diagram, the system can directly inspect the XML or the code to extract textual content, object roles, and styling. This can yield more accurate results (e.g. knowing the actual text string and font rather than relying on OCR from the image). It can also allow pinpointing issues more directly in the source (like identifying the line number in code where a color needs change). Thus, the pipeline design balances format-specific parsing with format-neutral image analysis. It first tries to use structured data if available; otherwise, it falls back to purely visual analysis. In all cases, the final decision criteria are the same numbers and thresholds.

Maintaining **deterministic rules** also means that the checks are not sensitive to minor style differences between formats. For instance, two different rendering engines might anti-alias text slightly differently – but the contrast check in our pipeline will sample enough pixels to compute a reliable ratio unaffected by one pixel's quirk. By focusing on robust metrics, we avoid false positives/negatives that could occur if the logic was naively tuned to one format's output.

Finally, the pipeline is extensible to new formats. If a new diagramming tool or format emerges, one only needs to add a converter to PNG (or to a common intermediate format) and possibly a parser for any meta-data. The validation rules themselves remain applicable. This future-proofs the system in a landscape of evolving visualization tools.

Conclusion

In summary, **Track A's research** delivers a comprehensive blueprint for a **Visual Diagram Quality Validation Pipeline** that is systematic, fast, and grounded in design best practices. By converting diverse inputs into a normalized form and rigorously checking against objective standards (accessibility, color usage, typography, spacing, etc.), the pipeline ensures that every diagram meets professional quality bars. It not only finds issues but also provides clear guidance to fix them, effectively acting like a **"diagram linter"** for creators. The approach emphasizes deterministic evaluations – every rule is backed by a measurable criterion such as "contrast $\geq$ 4.5:1" or "font $\geq$ 12pt" – avoiding any ambiguity in what is acceptable.

This solution addresses the discovered requirements: a **universal visual pipeline** from any format to PNG, **deterministic quality rules** covering consistency and WCAG accessibility, **enforcement of professional standards** (like typography and alignment), **validation of visual flow** and grouping, **color system checks** including colorblind safety, and even ensuring **model consistency** with the underlying data. It adapts to context, distinguishing between an internal technical diagram and an executive summary graphic, so the output is **appropriate for the audience** every time. All of this happens in near real-time (<5s), supporting an interactive feedback loop that improves diagrams at the point of creation.

By focusing on measurable design principles and avoiding subjective judgments, the pipeline provides trust and transparency in its assessments. Users can see exactly why an element is flagged and how to improve it, learning as they go. The end result is a higher baseline quality for all visuals produced – diagrams that

are not only aesthetically pleasing but also accessible, accurate, and tailored to their purpose. This systematic validation pipeline thus empowers teams to create **clear, effective, and professional diagrams** consistently, saving time in review cycles and elevating the communication value of their visual content.

Sources: The guidelines and rules implemented draw from established standards and expert recommendations, including accessibility criteria from WCAG [1] [3], colorblind-safe design practices [5] [8], and diagram design best-practices on layout and consistency [15] [18], among others, as cited throughout. Each rule is grounded in these accepted principles to ensure the pipeline's advice is credible and outcome-focused.

---

[1] [6] [7] How to Design for Color Blindness
https://www.audioeye.com/post/8-ways-to-design-a-color-blind-friendly-website/

[2] [10] [13] [15] Diagram Design: Tips for Effective Visual Diagrams | MiroBlog
https://miro.com/blog/diagram-design/

[3] [4] Informational Graphic Contrast (Minimum) - Low Vision Accessibility Task Force
https://www.w3.org/WAI/GL/low-vision-a11y-tf/wiki/Informational_Graphic_Contrast_(Minimum)

[5] [9] Guidelines color blind friendly figures | Netherlands Cancer Institute
https://www.nki.nl/about-us/responsible-research/guidelines-color-blind-friendly-figures

[8] SBFAQ Part 6: Color for Text, Sign and Graph Legibility
https://www.visualexpert.com/Resources/cfaqPart6.html

[11] How to Pick the Perfect Font Size: A Guide to WCAG Accessibility - The A11Y Collective
https://www.a11y-collective.com/blog/wcag-minimum-font-size/

[12] [14] [16] [17] [18] [19] Keep it simple & follow these flowchart rules for better diagrams | Nulab
https://nulab.com/learn/design-and-ux/keep-it-simple-follow-flowchart-rules-for-better-diagrams/

[20] [21] [22] Know Your Audience: making architecture diagrams valuable
https://www.orbussoftware.com/resources/blog/detail/know-your-audience-making-architecture-diagrams-valuable